

Rajalakshmi Engineering College

Name: M.S Nanthana shree
Email: 241901064@rajalakshmi.edu.in
Roll no: 241901064
Phone: 9360753427
Branch: REC
Department: CSE (CS) - Section 1
Batch: 2028
Degree: B.E - CSE (CS)

Scan to verify results



2024_28_III_OOPS Using Java Lab

2028_REC_OOPS using Java_Week 2_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Raj is solving a physics problem involving projectile motion, where he needs to calculate the time a ball hits the ground using a quadratic equation of the form $ax^2 + bx + c = 0$. Depending on the coefficients, the ball may hit the ground once, twice, or not at all in real time.

Help Raj find all real roots of the equation, if any.

Note: discriminant = $b^2 - 4ac$

Input Format

The input consists of three space-separated doubles a, b, and c, representing the coefficients of the quadratic equation.

Output Format

If there are two real roots, print:

- "Two real solutions:"
- "Root1 = <value>"
- "Root2 = <value>"

If there is one real root, print:

- "One real solution:"
- "Root = <value>"

If there are no real roots, print:

- "There are no real solutions."

Note: values are rounded to two decimal places.

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 1 6 9

Output: One real solution:

Root = -3.00

Answer

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        double a = sc.nextDouble();
        double b = sc.nextDouble();
        double c = sc.nextDouble();

        double discriminant = b * b - 4 * a * c;

        if (discriminant > 0) {
            double root1 = (-b + Math.sqrt(discriminant)) / (2 * a);
```

```

double root2 = (-b - Math.sqrt(discriminant)) / (2 * a);
System.out.printf("Two real solutions:\nRoot1 = %.2f\nRoot2 = %.2f",
root1, root2);
} else if (discriminant == 0) {
double root = -b / (2 * a);
System.out.printf("One real solution:\nRoot = %.2f", root);
} else {
System.out.println("There are no real solutions.");
}
}
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

Ted, the computer science enthusiast, has accepted the challenge of writing a program that checks if the number of digits in an integer matches the sum of its digits.

Guide Ted in designing and writing the code to solve this problem using a 'do-while' loop.

Input Format

The input consists of an integer N, representing the number to be checked.

Output Format

If the sum is equal to the number of digits, print "The number of digits in N matches the sum of its digits."

Else, print "The number of digits in N does not match the sum of its digits."

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 20

Output: The number of digits in 20 matches the sum of its digits.

Answer

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int N = sc.nextInt();
        int temp = N, sum = 0, digits = 0;

        do {
            sum += temp % 10; // add last digit
            temp /= 10;      // remove last digit
            digits++;        // count digits
        } while (temp > 0);

        if (sum == digits) {
            System.out.println("The number of digits in " + N + " matches the sum of its digits.");
        } else {
            System.out.println("The number of digits in " + N + " does not match the sum of its digits.");
        }
    }
}
```

Status : Correct

Marks : 10/10

3. Problem Statement

Noah is analyzing numbers within a given range $[A, B]$ and wants to calculate a special sum. For each number in the range, he calculates the product of its odd digits (ignoring even digits). If the number contains no odd digits, it is skipped. The sum of these products for all numbers in the range is the result.

Write a program to compute this sum.

Example

Input:

10 12

Output:

3

Explanation:

For 10, odd digits = 1, product = 1.

For 11, odd digits = 1, 1, product = $1 * 1 = 1$.

For 12, odd digits = 1, product = 1.

Total sum = $1 + 1 + 1 = 3$

Input Format

The input consists of two space-separated integers A and B, representing the inclusive range boundaries.

Output Format

The output prints a single integer representing the sum of the products of odd digits for all numbers in the range.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 10 12

Output: 3

Answer

```
import java.util.Scanner;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int A = sc.nextInt();  
        int B = sc.nextInt();
```

```

int totalSum = 0;

for (int num = A; num <= B; num++) {
    int temp = num;
    int product = 1;
    boolean hasOdd = false;

    while (temp > 0) {
        int digit = temp % 10;
        if (digit % 2 == 1) { // odd digit
            product *= digit;
            hasOdd = true;
        }
        temp /= 10;
    }

    if (hasOdd) {
        totalSum += product;
    }
}

System.out.println(totalSum);
}

```

Status : Correct

Marks : 10/10

4. Problem Statement

Maya, a student in an arts and crafts class, wants to create a pattern using stars (*) in a specific format. She plans to use a program to help her construct the pattern.

Write a program that takes an integer as input and constructs the following pattern using nested for loops.

Input: 5

Output:

*
* *
* * *
* * * *
* * * * *
* * * *
* * *
* *
*

Input Format

The input consists of a number (integer) representing the number of rows.

Output Format

The output displays the required pattern.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

Output: *

* *
* * *
* * * *
* * * * *
* * * *
* * *
* * *
* *
*
*

Answer

```
import java.util.Scanner;
```

```
public class Main {  
    public static void main(String[] args) {  
        Scanner sc = new Scanner(System.in);  
        int rows = sc.nextInt();  
  
        // Increasing part  
        for (int i = 1; i <= rows; i++) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
  
        // Decreasing part  
        for (int i = rows - 1; i >= 1; i--) {  
            for (int j = 1; j <= i; j++) {  
                System.out.print("* ");  
            }  
            System.out.println();  
        }  
    }  
}
```

Status : Correct

Marks : 10/10